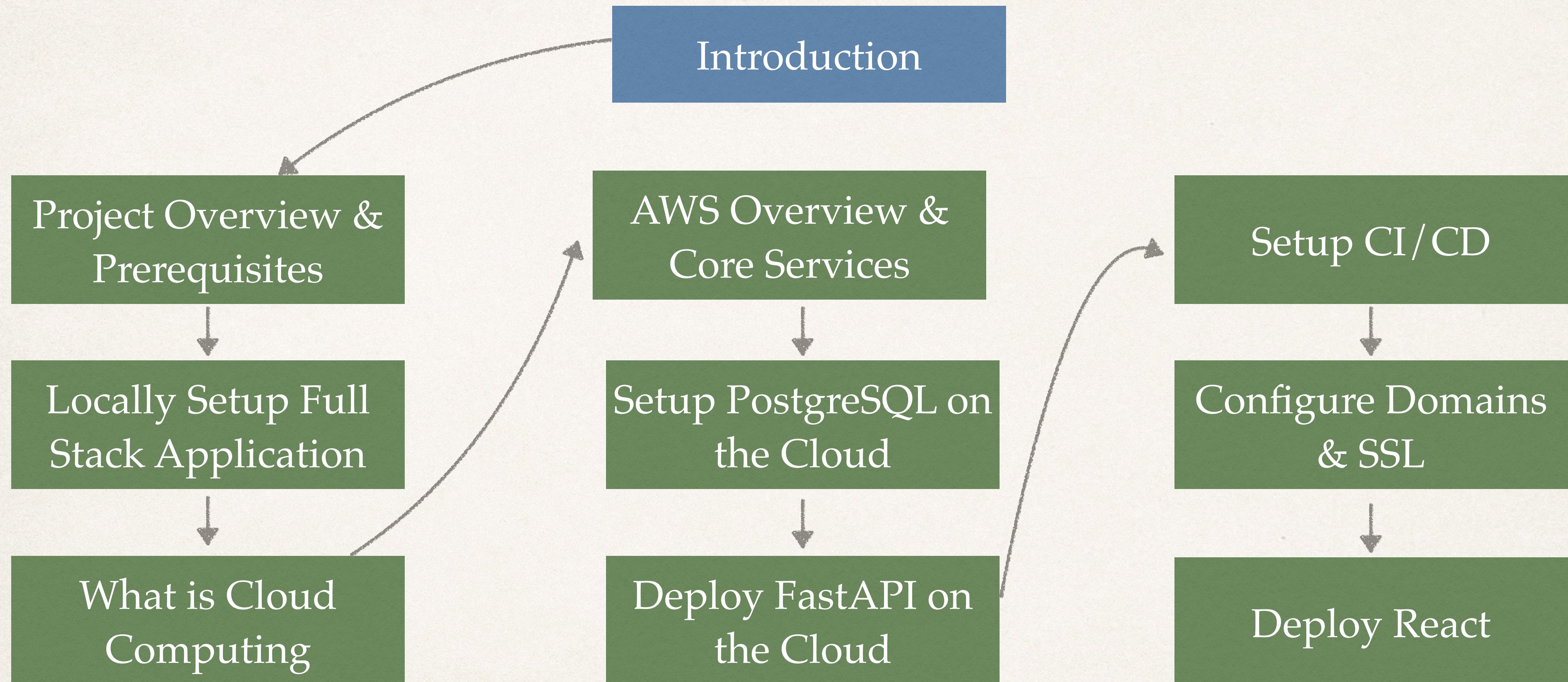


# Course Content





# Course Content!





# How To Get The Most Out Of This Course





# How To Get The Most Out Of This Course

Watch all the videos



Control the speed of videos to be  
your pace

Follow Along



Pause videos and follow along with  
each.

Search & Practice



Advance on your own and search  
for additional information

Ask Questions



Ask in Q&A section!



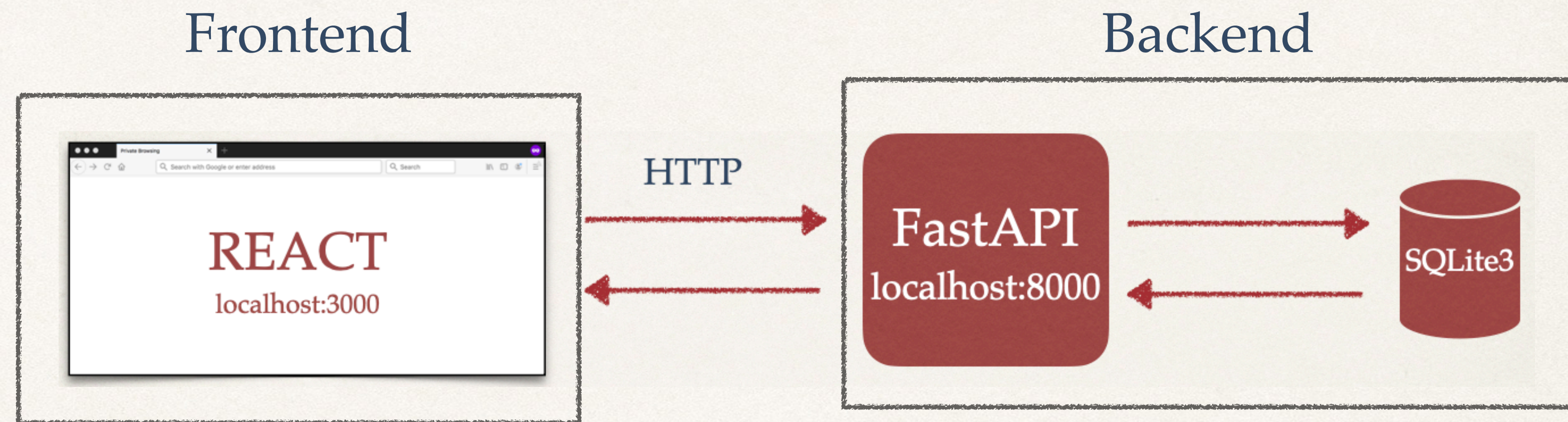
# Project Overview and Prerequisites





# Consists of 2 projects

- React application
  - HTML
  - CSS
  - JavaScript
- FastAPI
  - Python
  - SQLite3 database





# Project Overview

- We have a full stack application that uses:

- React
- FastAPI
- SQLite3

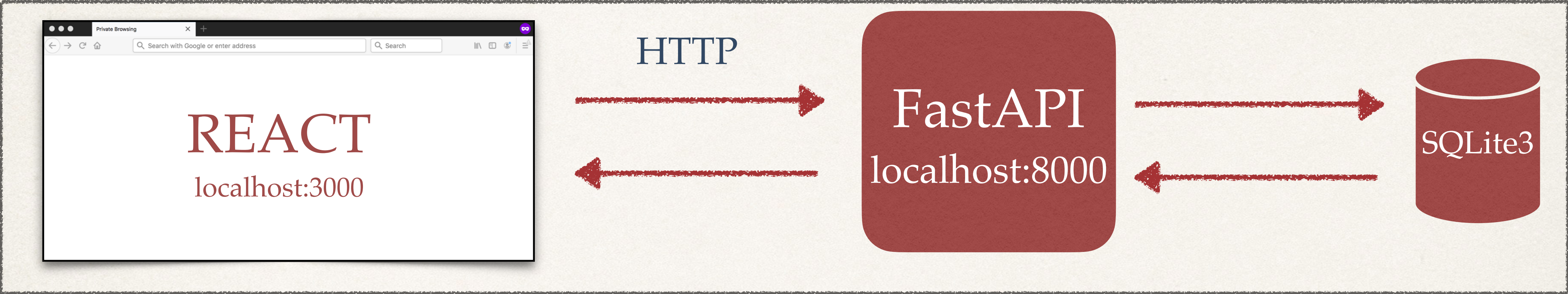
Dependencies



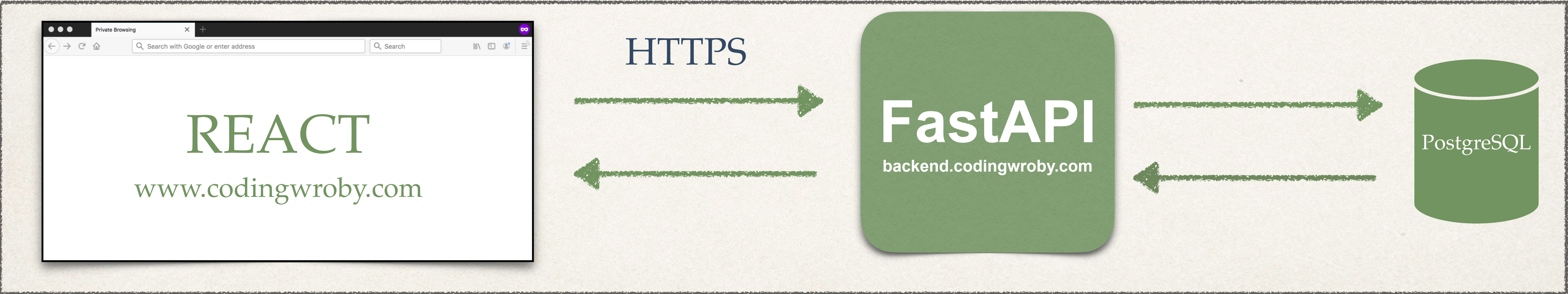


# Project Overview

Localhost



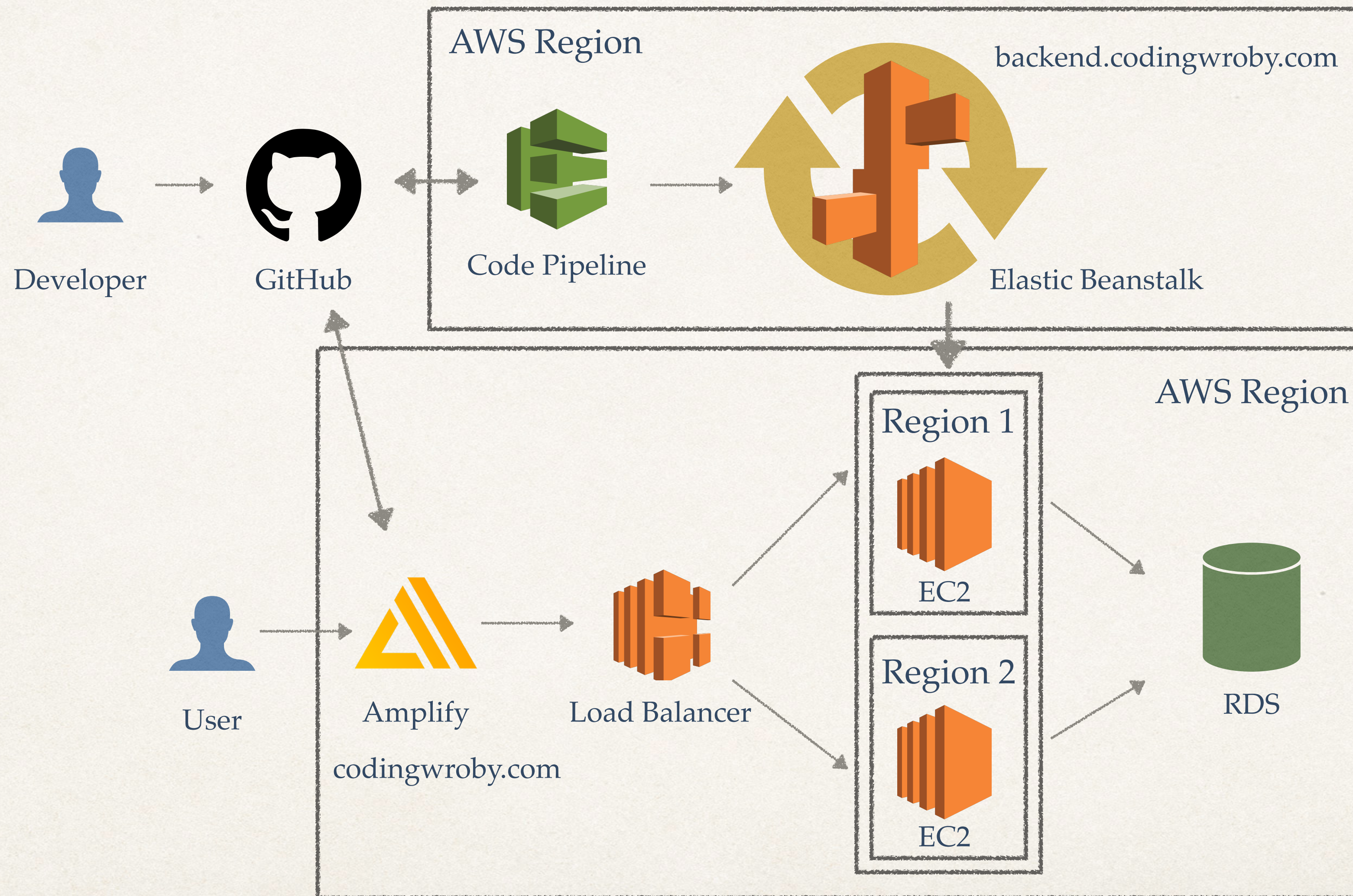
AWS





# AWS Architecture?

## Architecture





# Run Project Locally & Course Setup

- We really need 3 things to get the project running locally.

- Python Installed
- Node Installed
- Visual Studio Code Installed

- Git Installed
- GitHub account



# Is Python 3 Installed?

- Inside your “terminal” for Mac, or “command prompt” for windows.
- Type “python3 --version” for Mac or “python --version” for windows

```
ericroby@Eric's-MacBook-Pro ~% python3 --version  
Python 3.11.2
```



Could be a different version for you, requirement for FastAPI is version 3.7



# Is Node Installed?

- Inside your “terminal” for Mac, or “command prompt” for windows.
- Type “node -v” for Mac and windows

```
ericroby@Eric's-MacBook-Pro ~% node -v  
v21.0.0
```



Could be a different version for you



# Visual Studio Code?

- IDEs are simply a source code editor. This means an IDE will help and assist in writing software!
- Many of them have terminals and other useful build automation tools embedded within.
- IDEs have debugger tools that allow you to dive deep within the code to find bugs and create solutions

We will be using Visual Studio Code for our IDE



# Git & GitHub Account

- We will be using Git to manage our project's repositories.
- We will have two separate Git projects.
  - fastapi-aws
  - react-aws
- Both Git Repositories will be pushed onto GitHub.
- We are going to be implementing CI/CD between GitHub and AWS.



# Download Project

- Download project from link provided with course.



# React & FastAPI Installation

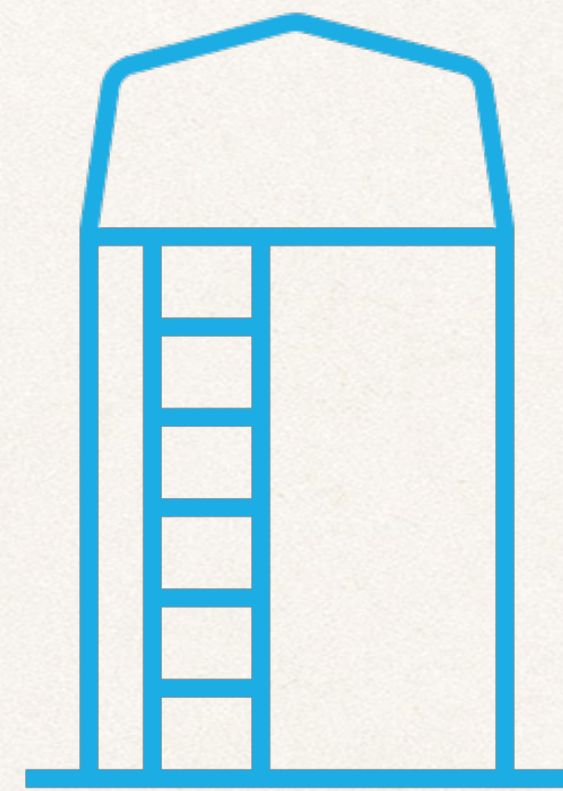
The logo features the text 'AWS' in white, bold, italicized sans-serif font, positioned above the text 'FULL STACK' in a bright green, bold, italicized sans-serif font. A white, stylized cloud icon is placed between the two text elements. The entire logo is set against a dark green rectangular background.

***AWS***   
***FULL STACK***

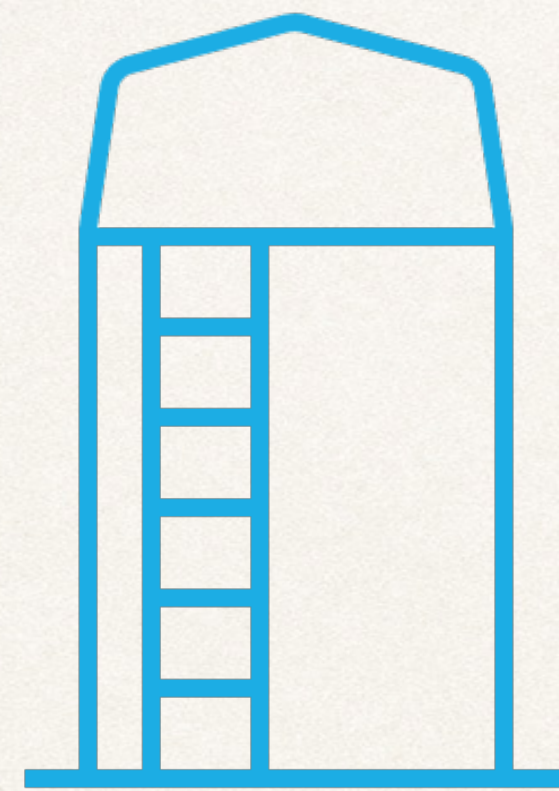


# Python Virtual Environments

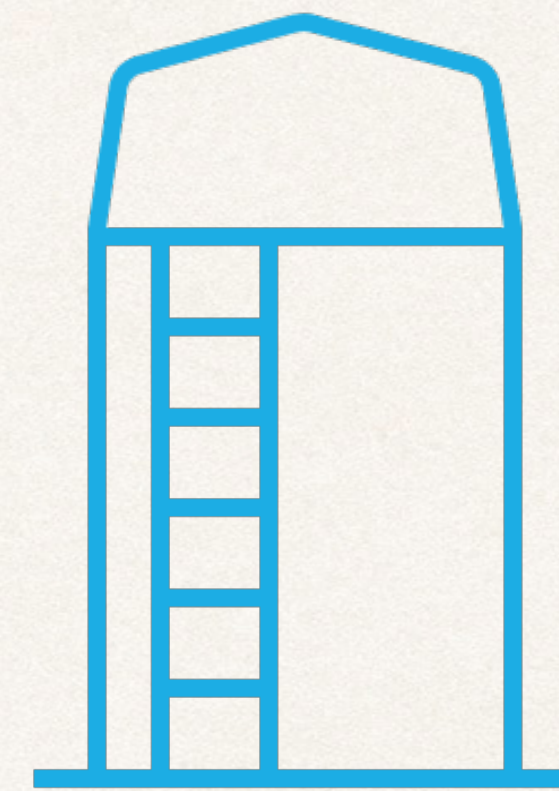
- A virtual environment is a Python environment that is isolated from those in other Python environments.



FastAPI



A.I.



Internet of Things



# How to Install Dependencies - FastAPI

- Pip is the Python package manager.
- Pip is used to install and update packages.
- You will want to make sure you have the latest version of pip installed.

```
ericroby@Ericks-MacBook-Pro ~% python3 -m pip --version  
pip 22.3.1
```



# Setting Up Virtual Environment - FastAPI

- Python already has an included dependency for virtual environments called venv.
- Creating a new FastAPI environment as a virtual environment
- Activating our virtual environment to install our dependencies in



# How to Install Dependencies - FastAPI

- We have a requirements.txt file which holds all the dependencies
- This file tells us what versions of each dependency we need to install
- We can easily install all dependency by doing:

```
ericroby@Eric's-MacBook-Pro ~% pip install -r requirements.txt  
<all dependencies will install here>
```



# How to Install Dependencies - React

- We will use npm (node package manager)
- `npm install`
- `npm i`

```
ericroby@Eric's-MacBook-Pro ~% npm install  
<all dependencies will install here>
```



# What is Cloud Computing?

The logo features the text 'AWS' in white, bold, italicized capital letters. To its right is a white, stylized cloud icon. Below 'AWS' and the cloud is the text 'FULL STACK' in a bright green, bold, italicized sans-serif font. The entire logo is set against a dark green rectangular background.

***AWS***   
***FULL STACK***



# What is cloud computing?

- Delivery of computing services over the internet
  - Including:
    - Storage
    - Networking
    - Database
    - Servers, and more..
  - Allows us to:
    - Rent computing power
    - Use without buying
    - Pay as you go model
    - Scale, and more..
- The cloud providers handles all the infrastructure.



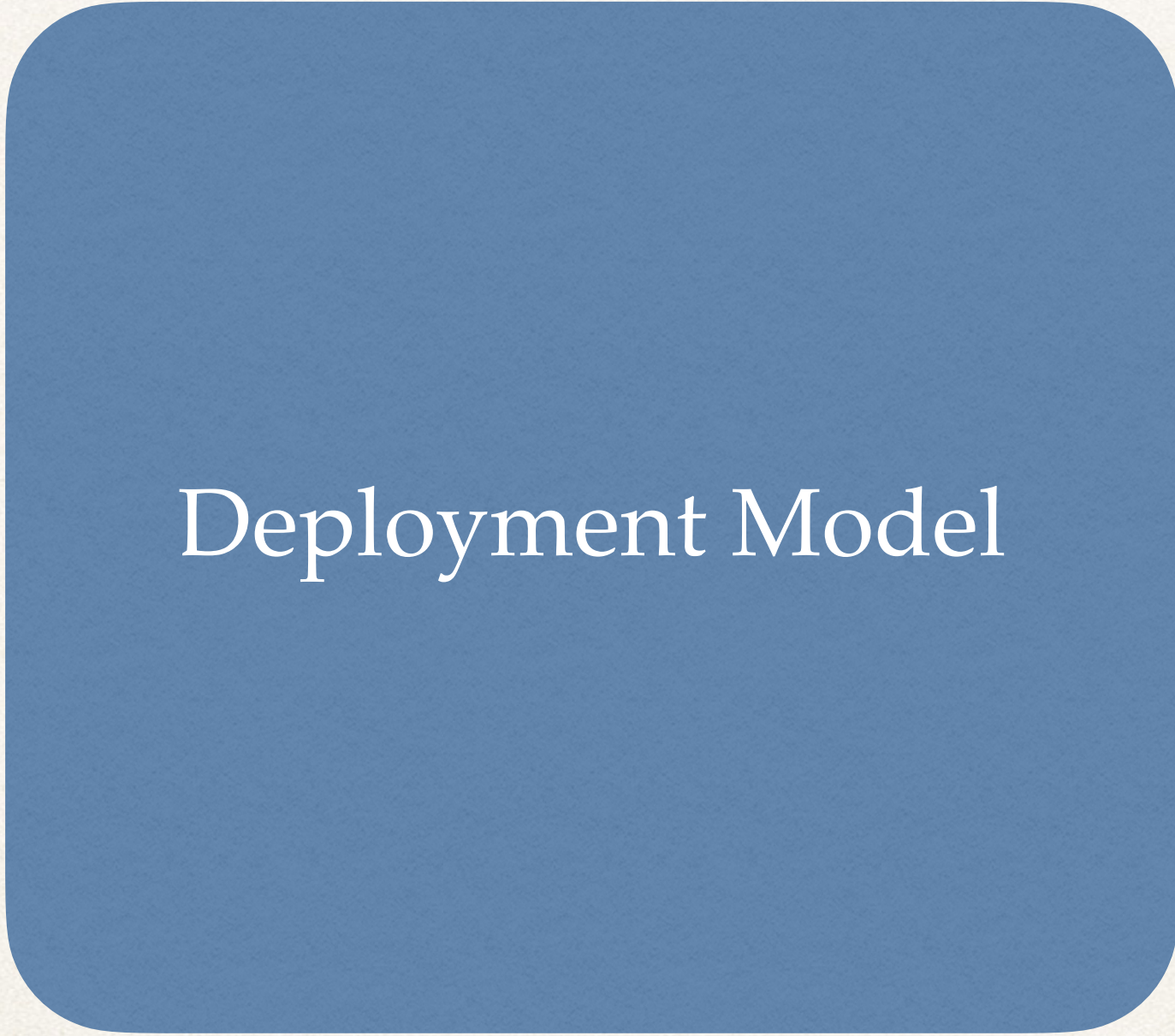
# Why choose to use cloud computing?

- Many benefits to use Cloud Computing
  - **Cost:** Reduces the costs of buying hardware and software, and setting up and running on-site data centers.
  - **Scalability:** Easily scales up or down based on demand.
  - **Performance:** Large economies of scale and efficient resource allocation optimize performance.
  - **Security:** Provides robust overall security, though it requires trust in the provider's security practices.



# What is cloud computing?

- Cloud computing has two distinct models:



Deployment Model



Service Model



# What is Deployment Model

- Can be broken down even more into 3 categories:

## Public Cloud:

- Owned by 3rd party representative.
- Given to the public over the internet

## Private Cloud:

- Operated by a single organization
- No one outside the company has access to it.

## Hybrid Cloud:

- Combination of both public and private.
- Use public for some items, private for others.



# What is Service Model

- Can be broken down even more into 3 categories:

## IaaS

- Highest level of control.
- Allows you to customize almost everything.
- Allows for networking, storage and computing resource

## PaaS

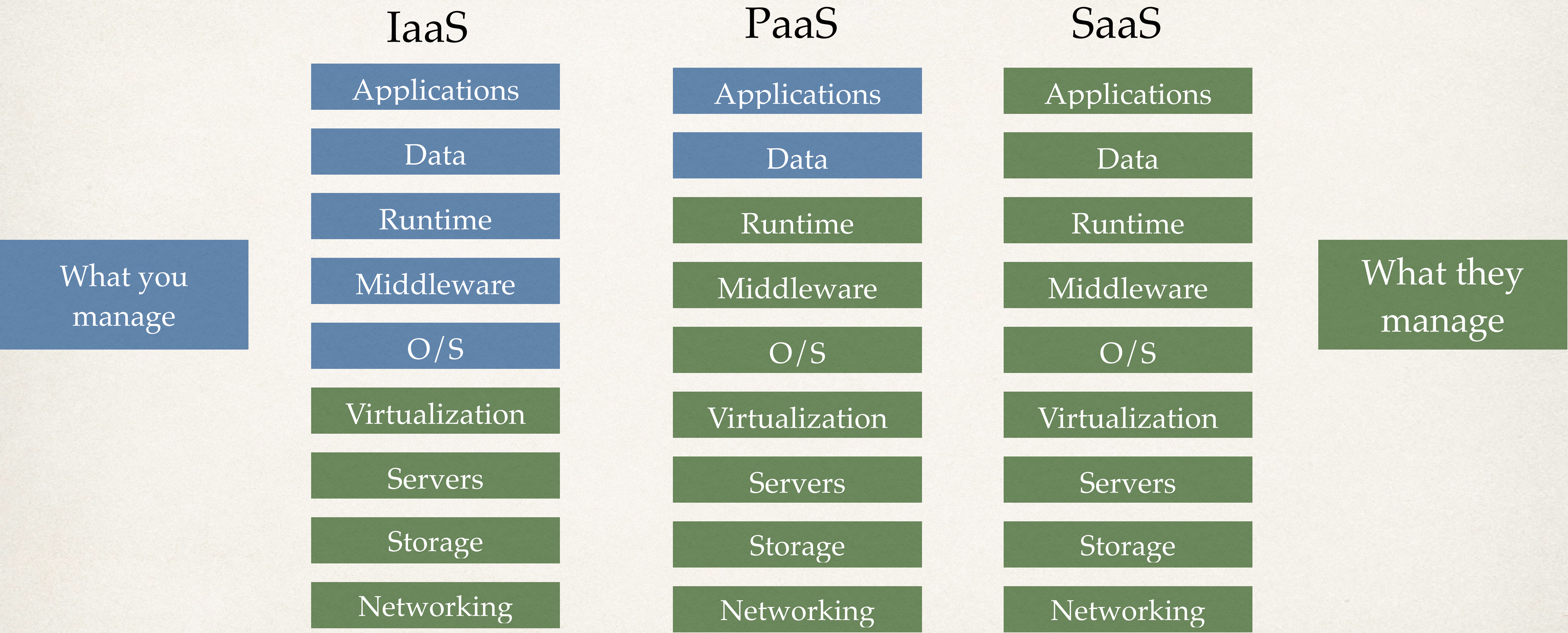
- Removes the user having to manage the infrastructure.
- This allows you to only have to worry about the code and how to deploy your application to the public.

## SaaS

- Product that is provided to you by the cloud provider.
- Everything is managed, you have to do the absolute least amount of work.



# What is Service Model





# AWS Overview





# What is AWS

- AWS stands for Amazon Web Service.
- AWS is a subsidiary of Amazon which provides on-demand cloud computing platforms and services.
- Started in 2006 but has changed a lot since then due to services and budget.
- AWS main role is to provide a scalable, efficient and cost-effective cloud computing solution.



# What is AWS

- AWS is a cloud computing provider
- AWS allows you to do online hosting (deployments).
- AWS allows you to create databases and use online storage.
- AWS allows you to use services that implement directly into your applications.

Leader in cloud computing



# How will we be using AWS?

- We will be using AWS for our:
  - Hosting & computing resources (scalability)
  - CI/CD (continuous integration & continuous deployments)
  - Deployment (make app live)
  - Storage (database)
  - Domain (URL)
  - SSL (certificates / HTTPS)



# How big is AWS?

- Really BIG!
- AWS has over 100+ services that are available for you to use.
- Some of these services include ML, AI, IoT, serverless solutions, and more.
- In this course we will focus on many core services.
- These core services are what majority of individuals want to learn how to use to deploy, maintain and scale web applications in the cloud.



# What services will we be using?

- I'll go over exactly the services like EC2, Beanstalk, Amplify, Route53 and more in a few videos.
- We will use the services that allow us to create on-demand resources on the internet.
- Spin up servers & databases.
- Connect AWS to GitHub
- And more.. so this course is packed with useful content.



# How much of this course is free?

- We will create a brand new developer account from scratch.
- Once your new account is created you'll be able to do most of this course absolutely FREE (for 12 months).
- The cost you will have is purchasing a domain, VPC IPv4 & Hosted Zones (Only required if you want more than just FastAPI deployed).
- Example: [www.codingwithroby.com](http://www.codingwithroby.com)
- We will need a custom domain to complete entire course, but we will be purchasing this from AWS (you can also transfer if you already own one).



# AWS is global

- AWS is global and allows you to deploy to many regions all over the world.
- This greatly allows you to reduce latency and bring the best experience possible to your users.





# Create Free AWS Account & Course Cost





# Create Free AWS Account

- Once we create a Free AWS Account we will get access to a bunch of AWS resources for absolutely free.
- There are some guidelines that can be found at:

<https://aws.amazon.com/free>

- Overall, this gives us enough free computing resources to deploy a FastAPI application + PostgreSQL (and for 12 months after account is created).

*Therefore, if you need a basic free FastAPI + PostgreSQL ONLY solution  
this is a great option (12 months)*



# AWS Services

- Amplify: FREE for 12 months
- Service for React deployment
- Route 53: Cost of domain purchase (~ \$14 a year)
- Custom Domain ([www.codingwroby.com](http://www.codingwroby.com))
- Hosted Zones: \$0.50 a month
- Subroutes ([www.backend.codingwroby.com](http://www.backend.codingwroby.com))
- Load Balancer: VPC IPv4: \$.005 x 2 / hour. (~ \$7.2 a month based on 30 days)
- Makes our FastAPI be able to have HTTPS (requirement for Amplify)

~\$8.87 a month  
(including the purchase of custom domain)  
For the first 12 months.



# AWS Core Services





# AWS Core Services that will be used.

- Budgets
- EC2
- S3
- ELB
- Health monitoring
- Elastic Beanstalk
- RDS
- Route 53
- Certificate Manager
- Amplify
- Code Pipeline

These are some of the most popular AWS services available today!



# AWS Core Services (Budget)



- Billing & Cost Management: This covers all the costs associated with the select month.
- We can create budgets inside the billing & cost management.
- We will create a “Zero Spend Budget” - That means we only want to use the Free Tier provided by our new AWS account and to be notified if anything costs 1 cent or more.



# AWS Core Services (EC2)



- EC2 stands for Elastic Compute Cloud.
- IaaS (Infrastructure as a Service)
- What are EC2? Well they consist of:

- Virtual Machines (EC2)
- Virtual Drives (EBS / Elastic Block Store)
- Auto Scaling Groups (ASG)

EC2 is the fundamental cloud computing resource from AWS



# AWS Core Services (EC2) Cont.



- Allows you to select OS: Linux, Windows or MacOS
- CPU: Computing Power
- RAM: Random Access Memory
- Storage: Elastic Block Store (EBS for storage)
- Public / Private Networking
- and more..
- But, we will go over how to setup our app for deployment.

t2.micro is the instance type we will use for free tier



# AWS Core Services (ELB)



- ELB stands for Elastic Load Balancing
- Automatically distribute incoming application traffic to your deployed application.
- Allows for scalability and traffic distribution.

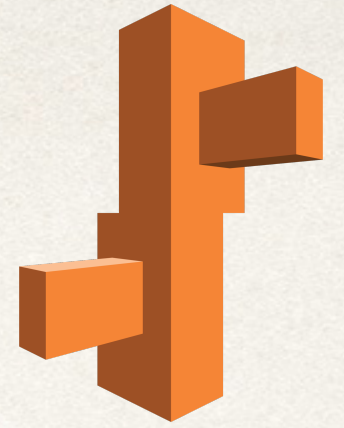


# AWS Core Services (Health Monitoring)

- Make sure our application is healthy and deployed.
- Reports when something is wrong, and when you need to take action.
- Helpful for when our app goes down or a service becomes unreliable.
  - Allows us to quickly take action.
  - Which helps keep our application running and available to all users.



# AWS Core Services (EBN)



- EBN stands for Elastic Beanstalk
- Allows us to automatically create the EC2s, ELB, S3 and Health Monitoring for our application.
- We will be utilizing EBN to create the environment we need to deploy our FastAPI application.
- Managed service (PaaS) by AWS.
- Free to use, but the resources created may not be free

Resources will be free from our AWS Free Tier plan



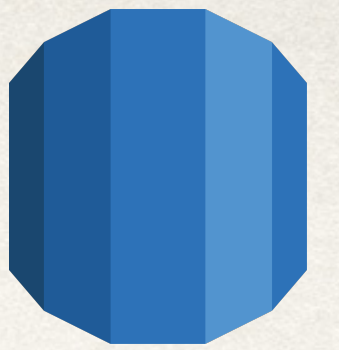
# AWS Core Services (Code Pipeline)



- CI/CD for our backend application (FastAPI).
- Will connect to our GitHub account to find new pushes to the head.
- Automatic deployments to our Elastic Beanstalk.
- Easily to configure and setup.



# AWS Core Services (RDS)



- RDS stands for Relational Database Storage.
- Managed database service for using SQL
- Allows you to create databases in the cloud and managed by AWS
- Since it is managed by AWS, they will handle provisioning.
- OS Updates
- Can also do backups, replications, scalability and more!

We will deploy RDS PostgreSQL on AWS



# AWS Core Services (Route53)



- Route users to your site by global Domain Name System (DNS) servers.
- Setup DNS routing with domain name registration.
- Customize policies for the DNS.
- Add sub-routes for our DNS.



# AWS Core Services (ACM)



- ACM stands for AWS Certificate Manager
- Manage and deploy SSL/TLS certificates.
  - This makes your app go from HTTP -> HTTPS
- ACM removes the process of uploading & renewing certificates.



# AWS Core Services (Amplify)



- Deploy our SPA applications efficiently.
- Allow us to implement CI/CD for our front end application.
- Store front end environmental variables.
- Scale for users.
- Works with React.



# Create AWS Budget





# AWS Core Services (Bill & Cost)



- Billing & Cost Management: This covers all the costs associated with the select month.
- We can create budgets inside the billing & cost management.
- We will create a “Zero Spend Budget” - That means we only want to use the Free Tier provided by our new AWS account and to be notified if anything costs 1 cent or more.
- The only time we may go over this is when we purchase our domain example: [codingwroby.com](https://codingwroby.com), hosted zones and setup a load balancer.



# AWS Core Services (Bill & Cost)



Overview [Info](#)

Budgets (1) [Info](#) [Download CSV](#) [Actions](#) [Create budget](#)

Type - Show all budgets ▼ < 1 > ⚙

| <input type="checkbox"/> | Name                                 | ▲ | Thresholds ▼   | Budget | Amount used | Forecasted amount | Current vs. budgeted ▼       | Forecasted vs. budgeted ▼     |
|--------------------------|--------------------------------------|---|----------------|--------|-------------|-------------------|------------------------------|-------------------------------|
| <input type="checkbox"/> | <a href="#">My Zero-Spend Budget</a> |   | ⚠ Exceeded (1) | 0.01   | \$0.03      | \$0.19            | <div><div></div></div> 2.50% | <div><div></div></div> 19.10% |



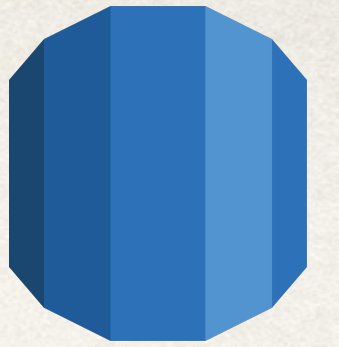


# What is RDS





# AWS Core Services (RDS)

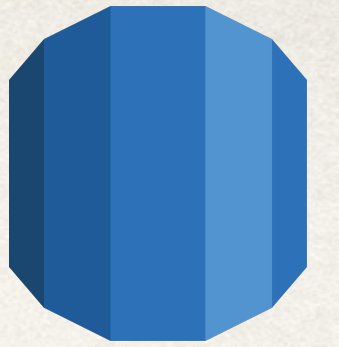


- RDS stands for Relational Database Storage.
- Managed database service for using SQL
- Allows you to create databases in the cloud and managed by AWS
- Since it is managed by AWS, they will handle provisioning.
- OS Updates
- Can also do backups, replications, scalability and more!

We will deploy RDS PostgreSQL on AWS



# Why use RDS?

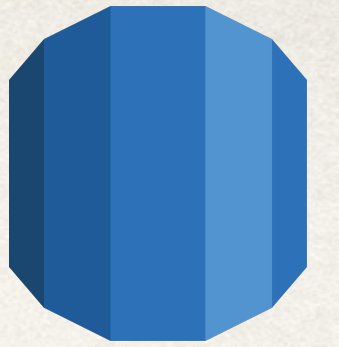


- AWS handling provisioning and OS updates
- Setup automatic backups
- Auto scale the size and resources
- Monitoring and maintenance
- Replications and failover
- Encryption

All makes RDS a great choice for  
our production DB



# Other options..

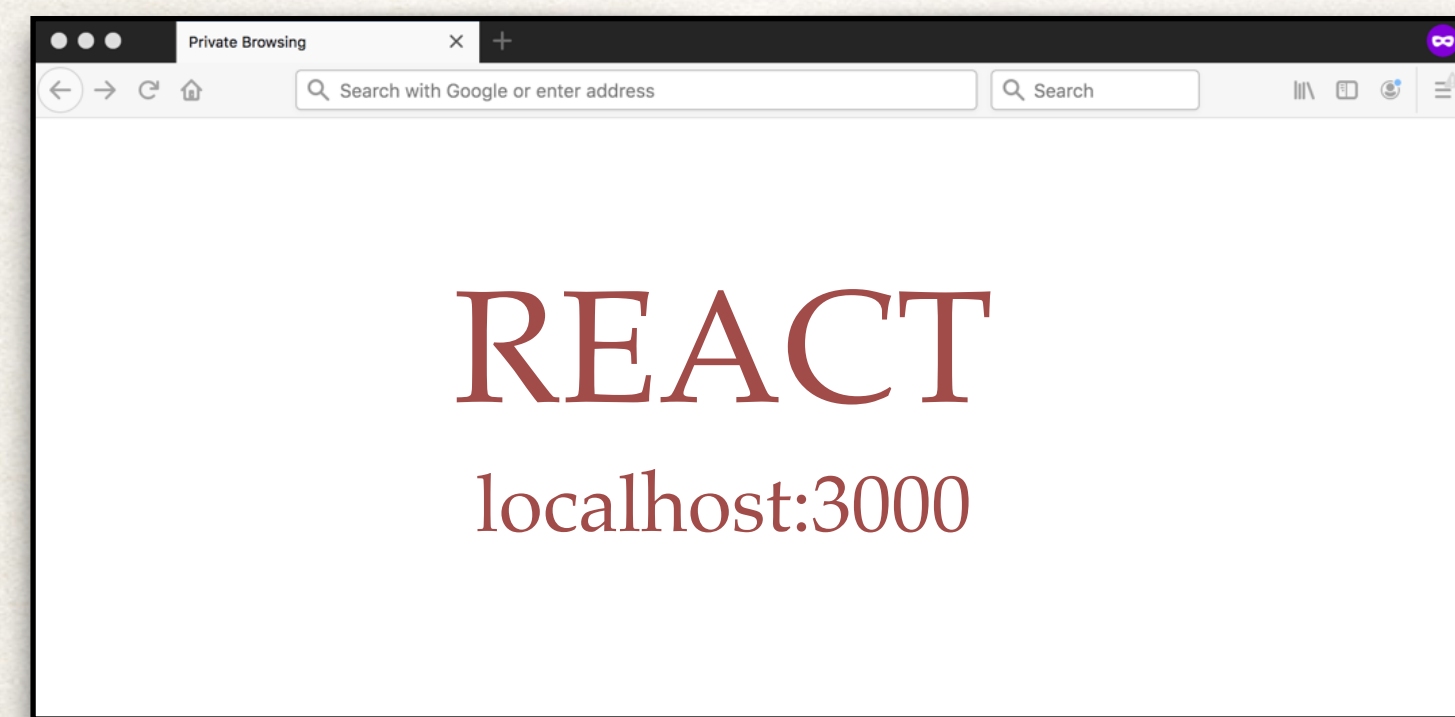
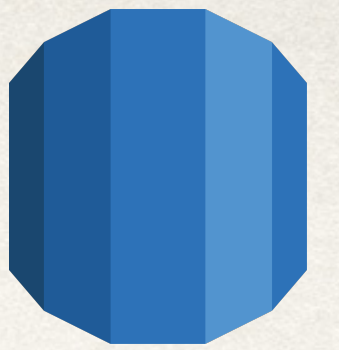


- If you didn't want AWS to handle your database infrastructure
- You could create an EC2
  - Deploy your database on the EC2
  - Handle all upgrades yourself
  - Handle all security yourself
  - Handle all backups yourself

Sounds like a lot of work!  
Therefore, lets use AWS!



# Project Overview



HTTP



AWS RDS



PostgreSQL username /  
password / endpoint

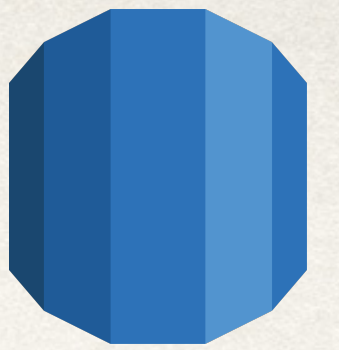


# Configure AWS Database to Local Project





# Connect to our RDS PostgreSQL



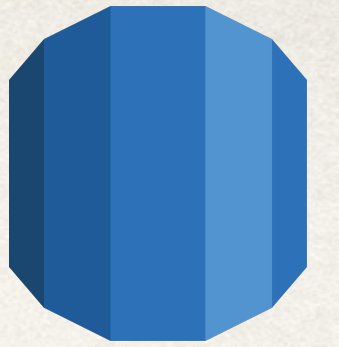
- Now we want to connect our local FastAPI application to RDS
- This is to test the connection before we deploy FastAPI
- To do this we need a to create the connection string:

```
"postgresql://<username>:<password>@<endpoint_name>:<port_name>/<db_name>"
```

- Okay, where do we find this information?



# Username and password



- Username can be found:
  - Inside RDS -> Configuration
- Password does not show:
  - You need to remember database password

Availability

Master username  
rootuser

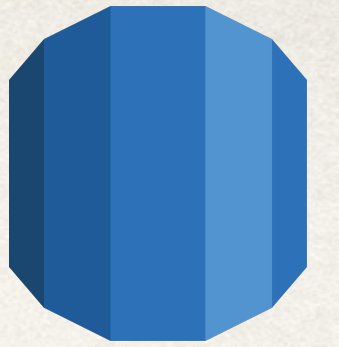
Master password

\*\*\*\*\*

`"postgresql://rootuser:test1234!@<endpoint_name>:<port_name>/<db_name>"`



# Endpoint



- Endpoint can be found:
  - Inside RDS -> Connectivity & Security
- Port can be found right underneath.
- Database name by default is:
  - postgres

Endpoint

 fastapi-aws-database.cbyus6448b9i.us-east-2.rds.amazonaws.com

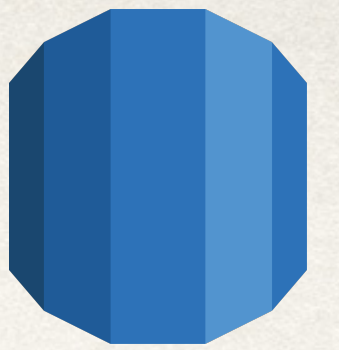
Port

5432

`"postgresql://rootuser:test1234!@fastapi-aws-database.:5432/postgres"`



# Connect to our RDS PostgreSQL



- After our string is created it should look like this:

```
"postgresql://<username>:<password>@<endpoint_name>:<port_name>/<db_name>"
```



```
"postgresql://rootuser:test1234!@fastapi-aws-database..:5432/postgres"
```



FastAPI

```
engine = create_engine("postgresql://rootuser:test1234!@fastapi-aws-database..:5432/postgres")
```

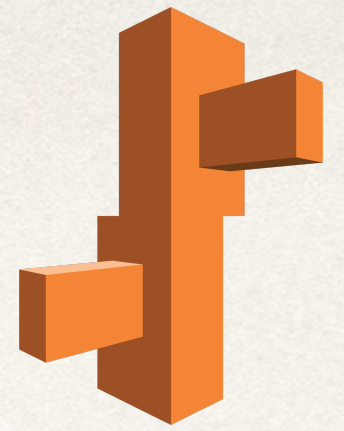


# Deploy FastAPI Overview

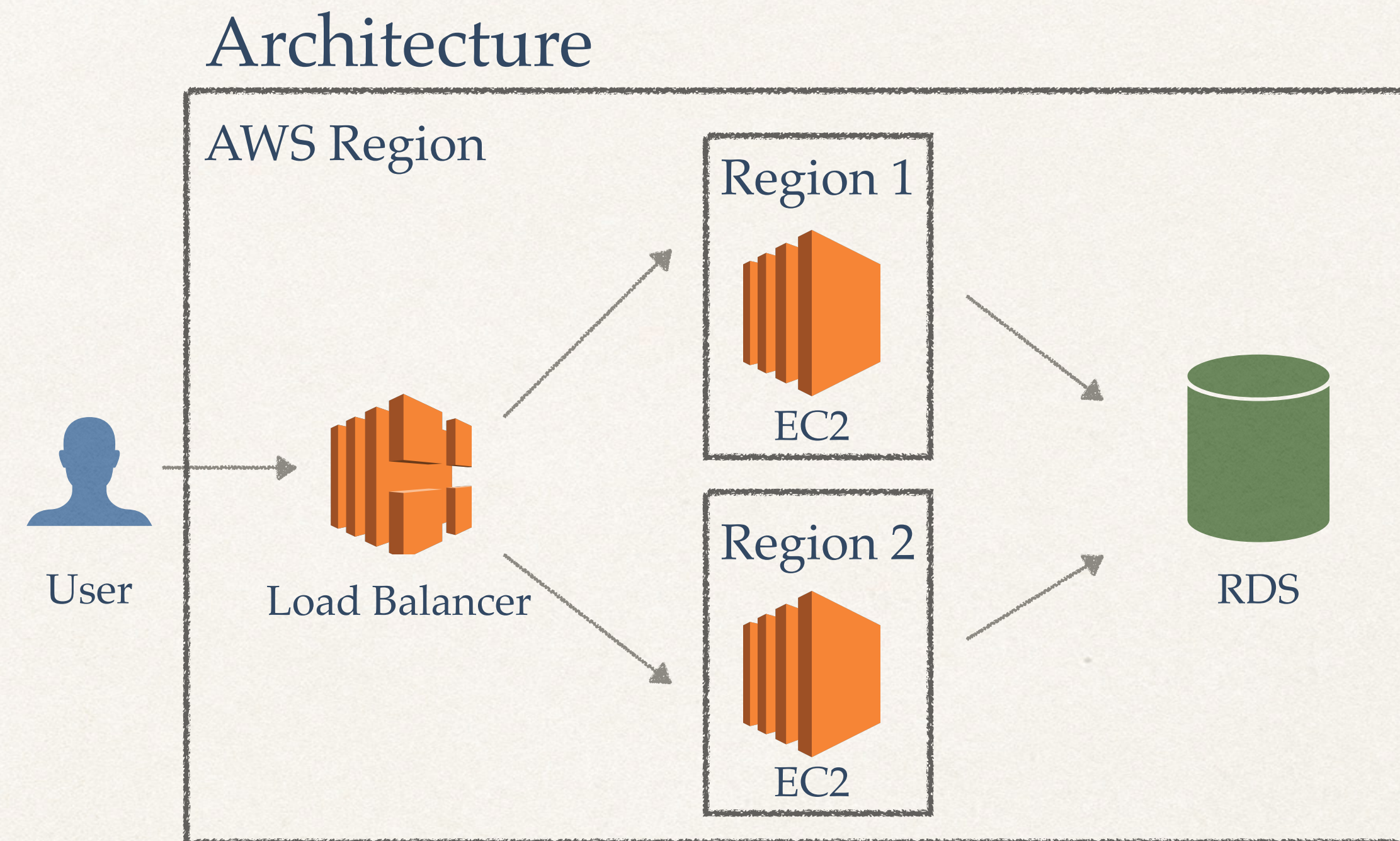




# How to do this?

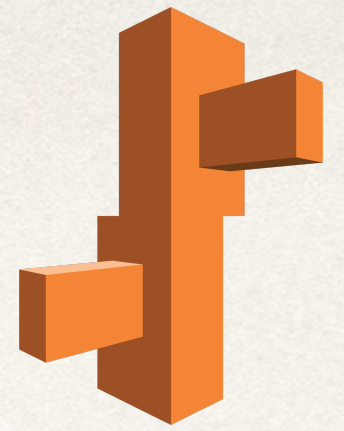


- To start from scratch:
- EC2
- Load Balancer





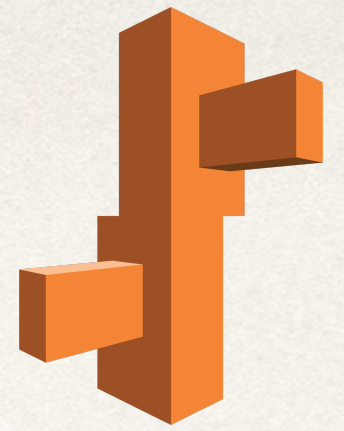
# AWS Core Services (EBN)



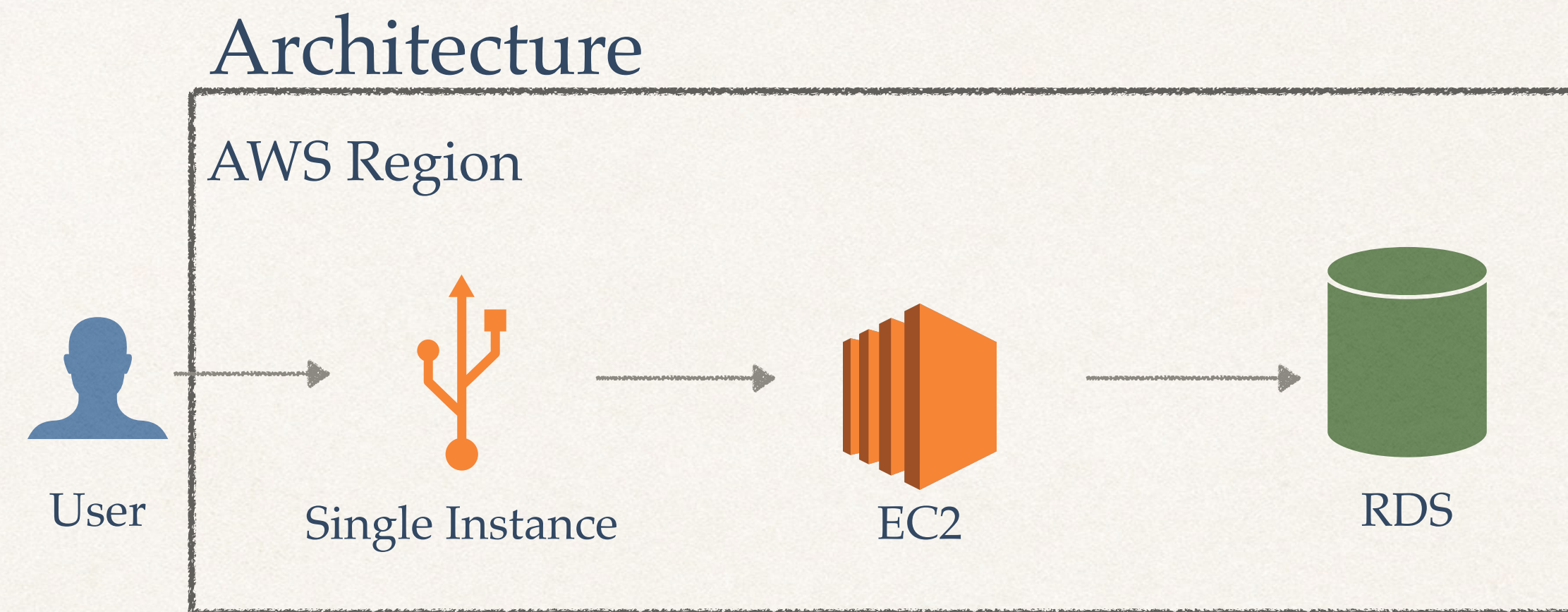
- EBN stands for Elastic Beanstalk
- Allows us to automatically create the EC2s, ELB, S3 and Health Monitoring for our application.
- We will be utilizing EBN to create the environment we need to deploy our FastAPI application.
- Managed service (PaaS) by AWS.
- Free to use, but the resources created may not be free



# AWS Core Services (EBN)



- What will Elastic Beanstalk create for us?



HTTP is considered Not Secure and your browser may tell you. We will fix this later on.

- By default: We will get a AWS URL that is HTTP not a HTTPS.

`http://fastapi-aws-us-2.elasticbeanstalk.com`



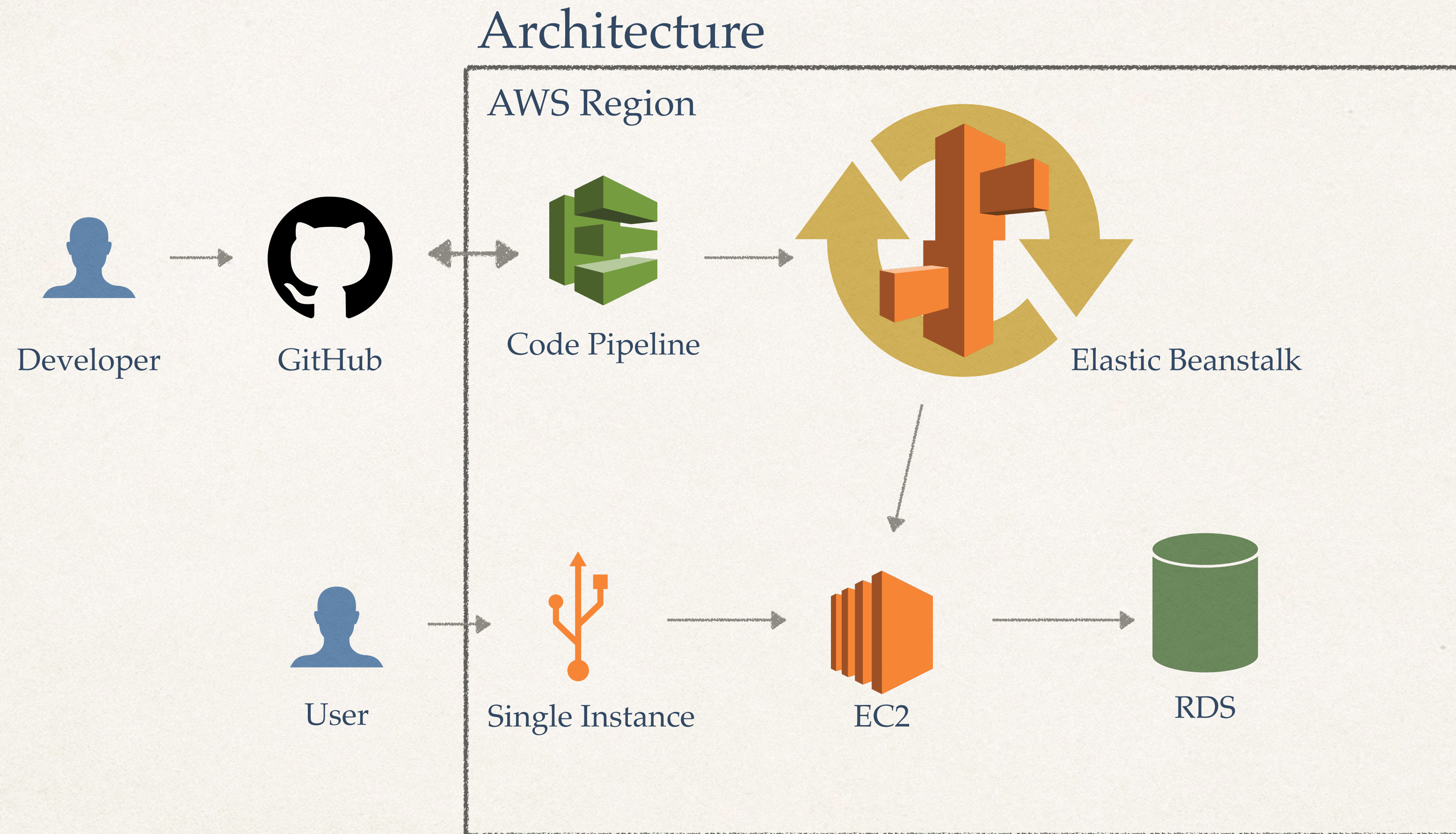
# AWS Core Services (Code Pipeline)



- CI/CD for our backend application (FastAPI).
- Will connect to our GitHub account to find new pushes to the head.
- Automatic deployments to our Elastic Beanstalk.
- Easily to configure and setup.

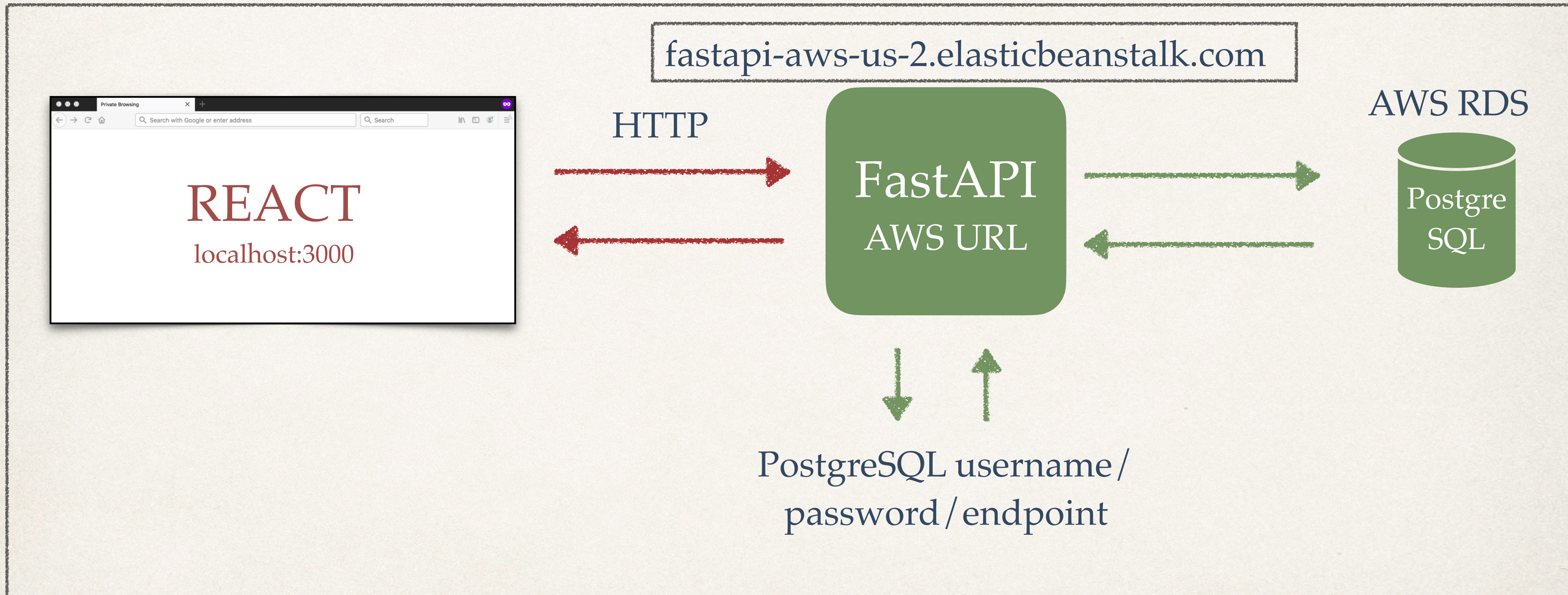
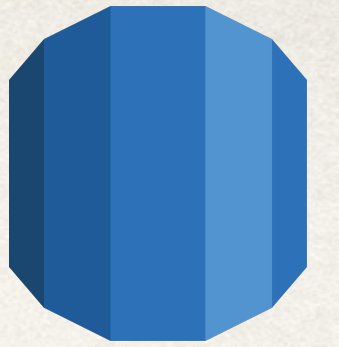


# AWS Core Services (Code Pipeline)





# Project Overview





# Route 53





# AWS Core Services (Route53)



- Route users to your site by global Domain Name System (DNS) servers.
- Setup DNS routing with domain name registration.
- Customize policies for the DNS.
- Add sub-routes for our DNS.

Will purchase [codingwroby.com](https://codingwroby.com)

Cost \$14 USD in video for 1 year



# AWS Core Services (Route53)



- When you register a domain you can first check for availability.

## Not Available

**Search for domain**

Check availability for a domain

× Search

**Search result**

 **codingwithroby.com is not available**  
See other available domains below.

## Available

**Search for domain**

Check availability for a domain

× Search

**Search result**

| Domain                                        | Price/year | Actions                |
|-----------------------------------------------|------------|------------------------|
| ericcodesyouknow.com <span>Exact match</span> | 14.00 USD  | <a href="#">Select</a> |

Only need 1 domain for the entire course



# AWS Core Services (Route53)



- After purchasing a domain it is not available immediately.

| Operation ID                         | Domain name     | Message | Status      | Type            |
|--------------------------------------|-----------------|---------|-------------|-----------------|
| 3d33e1ee-a3e6-4398-aeb7-32ddd53c9b40 | codingwroby.com | -       | In progress | Register domain |

- Email within 48 hours (usually ~15 minutes)
- Within email there will be a link
- Once clicked the status will change.



# Hosted Zones

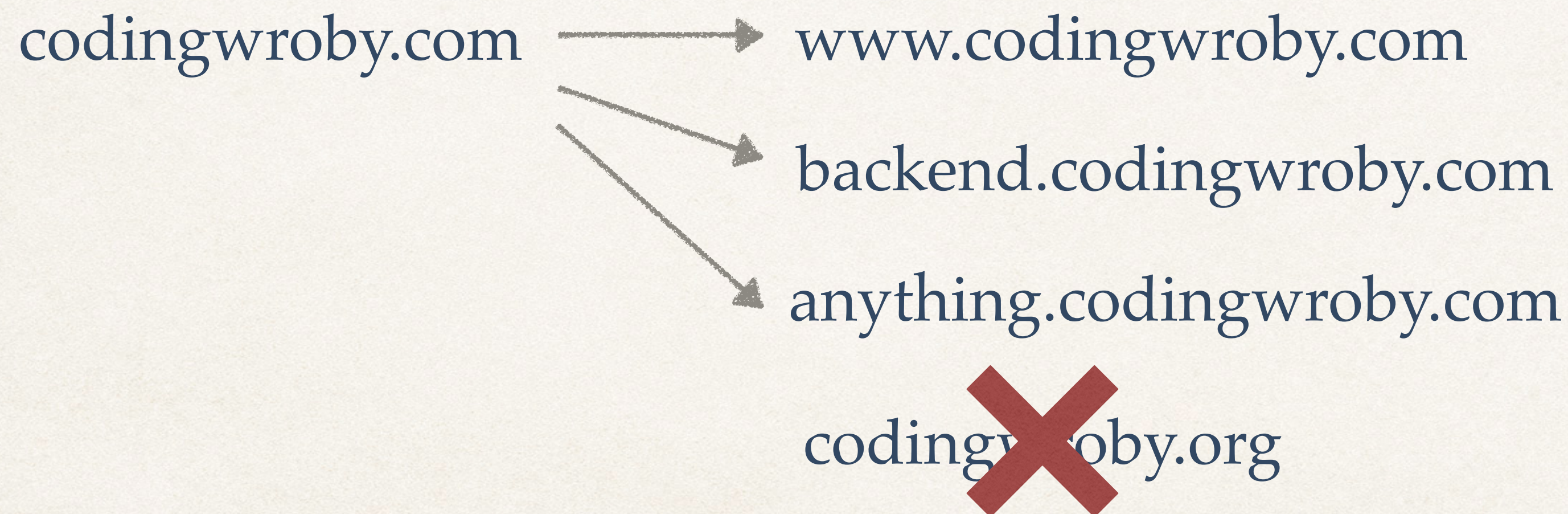




# Route53 (Hosted Zones)



- Allows us to create a collection of records that can be managed together.
- All of these records point to the same parent.
- Ability to create subdomains.





# Route53 (Hosted Zones)

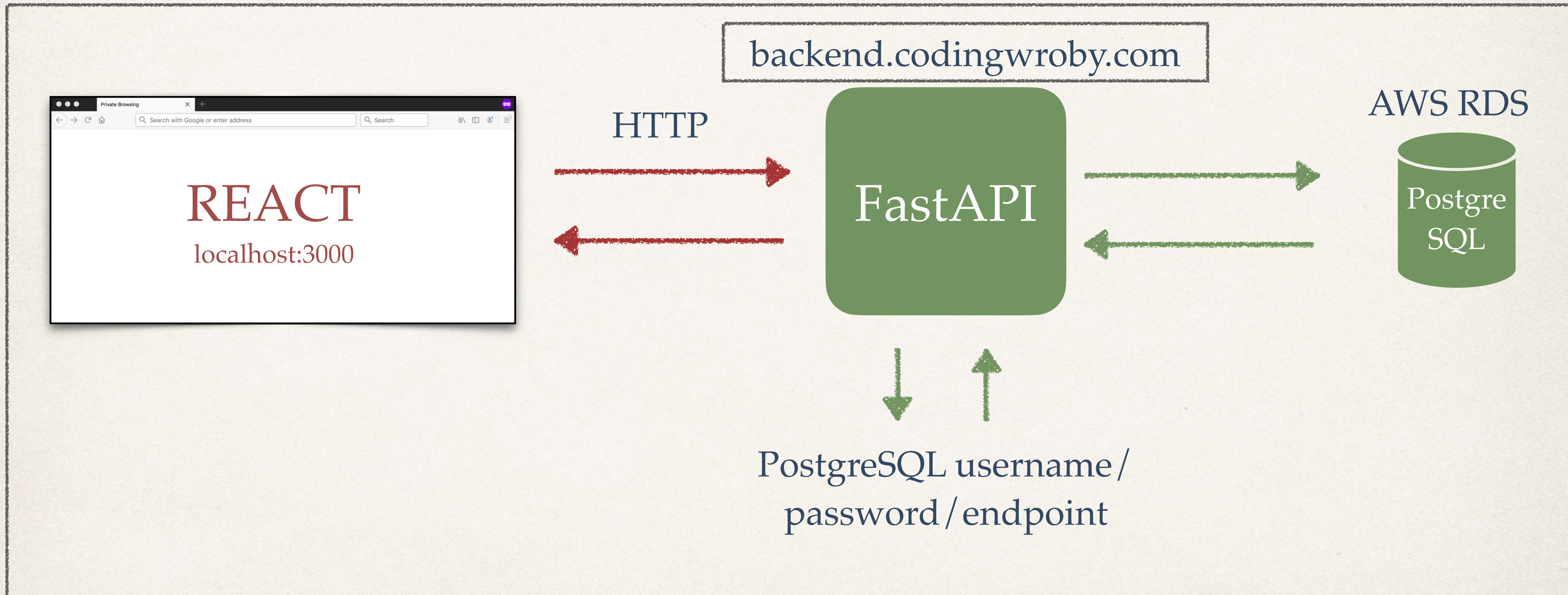


- Create new record name: backend.codingwroby.com
- Create a new record type: A
- Create an alias to route backend.codingwroby.com to fastapi-aws-us-2.elasticbeanstalk.com
- Routing policy: Simple

A screenshot of the AWS Route53 'Create record' console interface. The page has a light gray header with 'Create record' and an 'Info' link. Below the header is a 'Quick create record' section with a 'Switch to wizard' link. The main form area is titled 'Record 1' and includes a 'Delete' button. It contains fields for 'Record name' (backend) and 'Record type' (A). Below these is a toggle for 'Alias' which is turned on. The 'Route traffic to' section has three dropdowns: 'Alias to Elastic Beanstalk environment', 'US East (Ohio)', and a search bar containing 'fastapi-aws-us-east-2.elasticbeanstalk.com'. The 'Routing policy' is set to 'Simple routing'. There is also a toggle for 'Evaluate target health' which is turned off. At the bottom right is an 'Add another record' button.



# Project Overview





# Certificate Manager





# Certificate Manager

- Creates SSL / TLS Certificates
- Enable websites to use HTTPS which is more secure than HTTP.
- Makes SSL / TLS encryption possible.
- Therefore, this is a protocol for encrypting internet traffic and verifying server identify.
- Certificate manager makes it easy for use to get the SSL / TLS certificates for our domain.



# Certificate Manager

- What info does a SSL certificate need?
  - The domain name
  - Person or organization the domain is issued to
  - Certificate authority
  - Associated subdomains
  - Issue date
  - Expiration date

Since we got our domain on Route53 - AWS makes this very easy for us.



# Certificate Manager

- Sites we want certificates for:
  - codingwroby.com
  - backend.codingwroby.com
  - \*.codingwroby.com

\* is a wildcard meaning all subdomains.



# CNAME

- We need to adjust these hosted zones URLs to be CNAMEs to our route53 domain.
- CNAME (canonical name) record points to an alias domain.
- CNAME are used in lieu of an A record (which we already made).
- CNAME must point to a domain, never an IP address.
- We will setup CNAME in certificate manager.



# backend.codingwroby.com

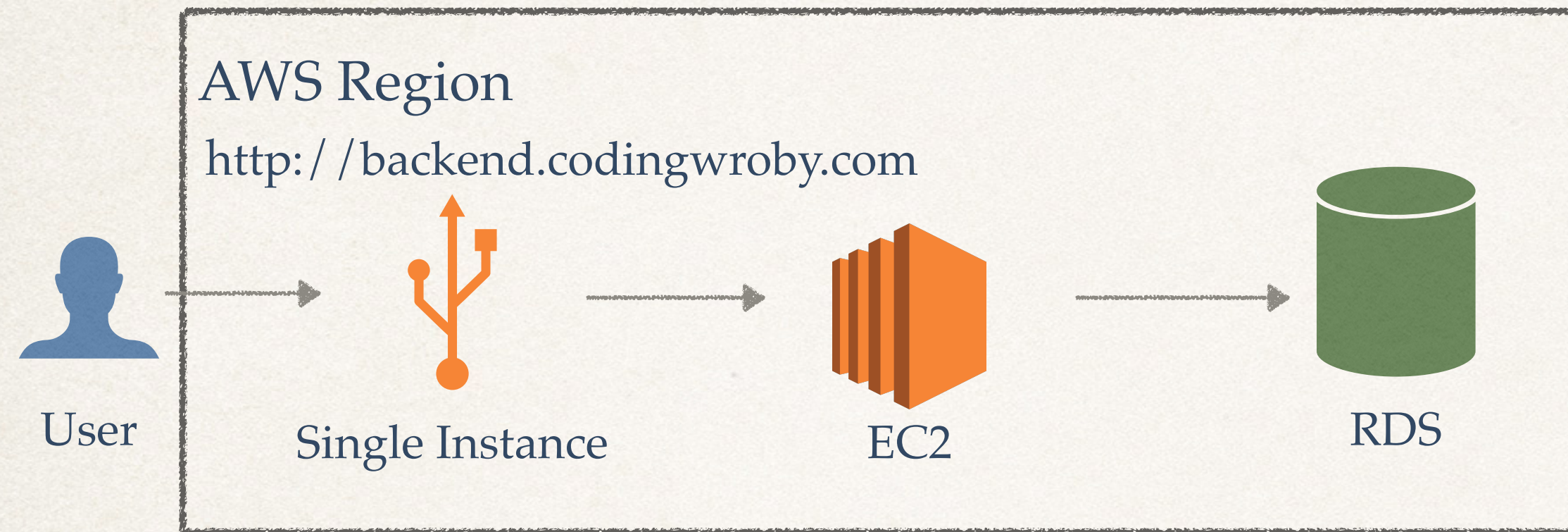
- After all the certificate manager setup our elastic beanstalk will still only work with HTTP not HTTPS.
- There is two reasons for this.
  - 1. Our EC2 rules does not yet allow for HTTPS only HTTP
  - 2. EC2 single instance does not allow HTTPS. We need to configure a load balancer.

Lets look at the architecture again.

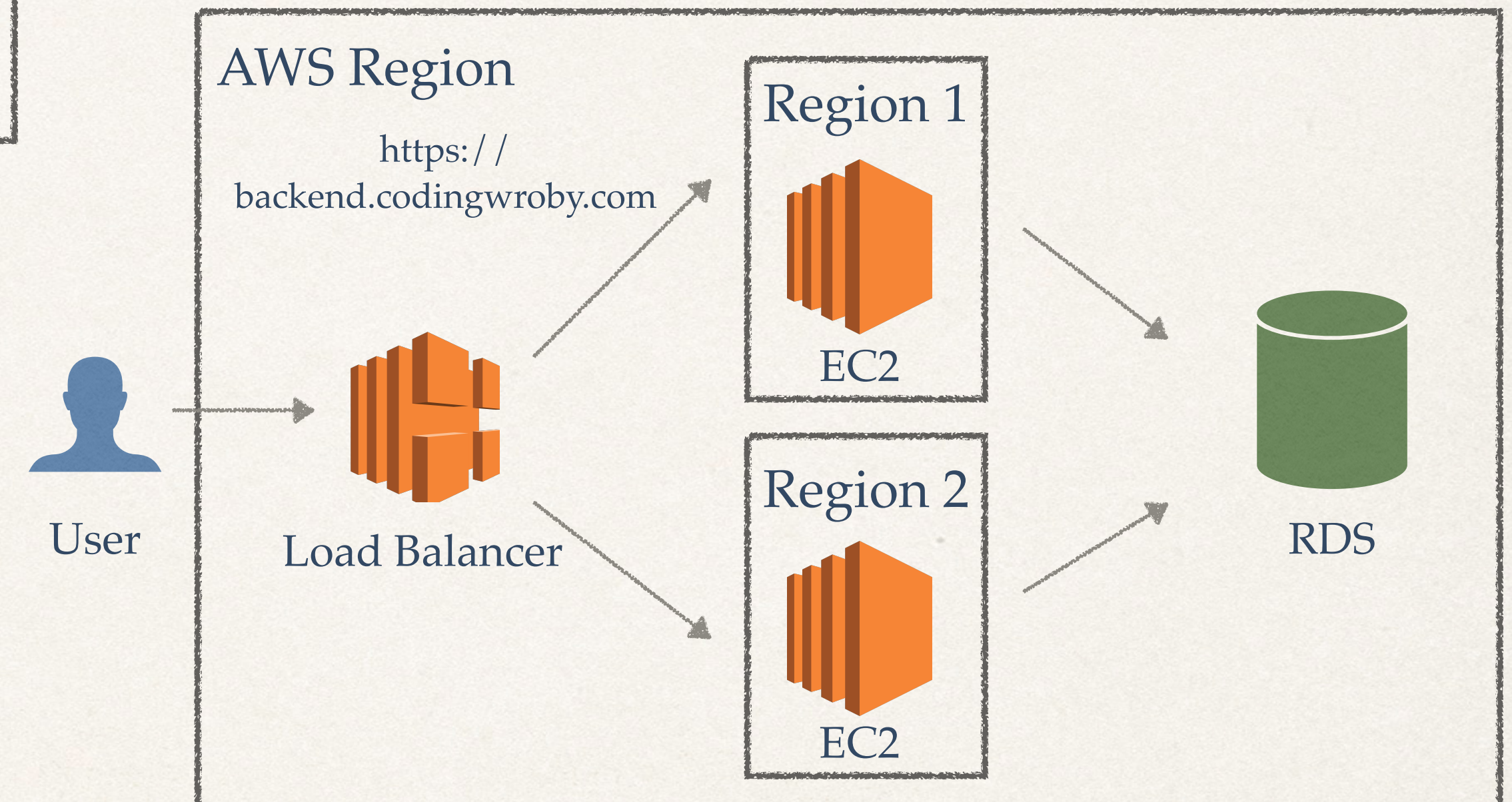


# Architecture

## Architecture



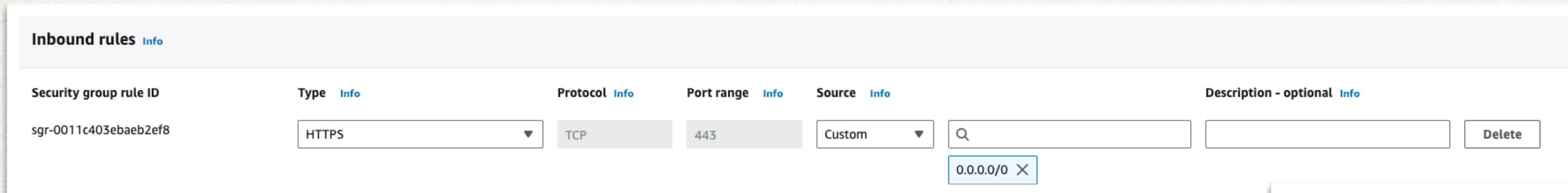
## Architecture





# How to do this?

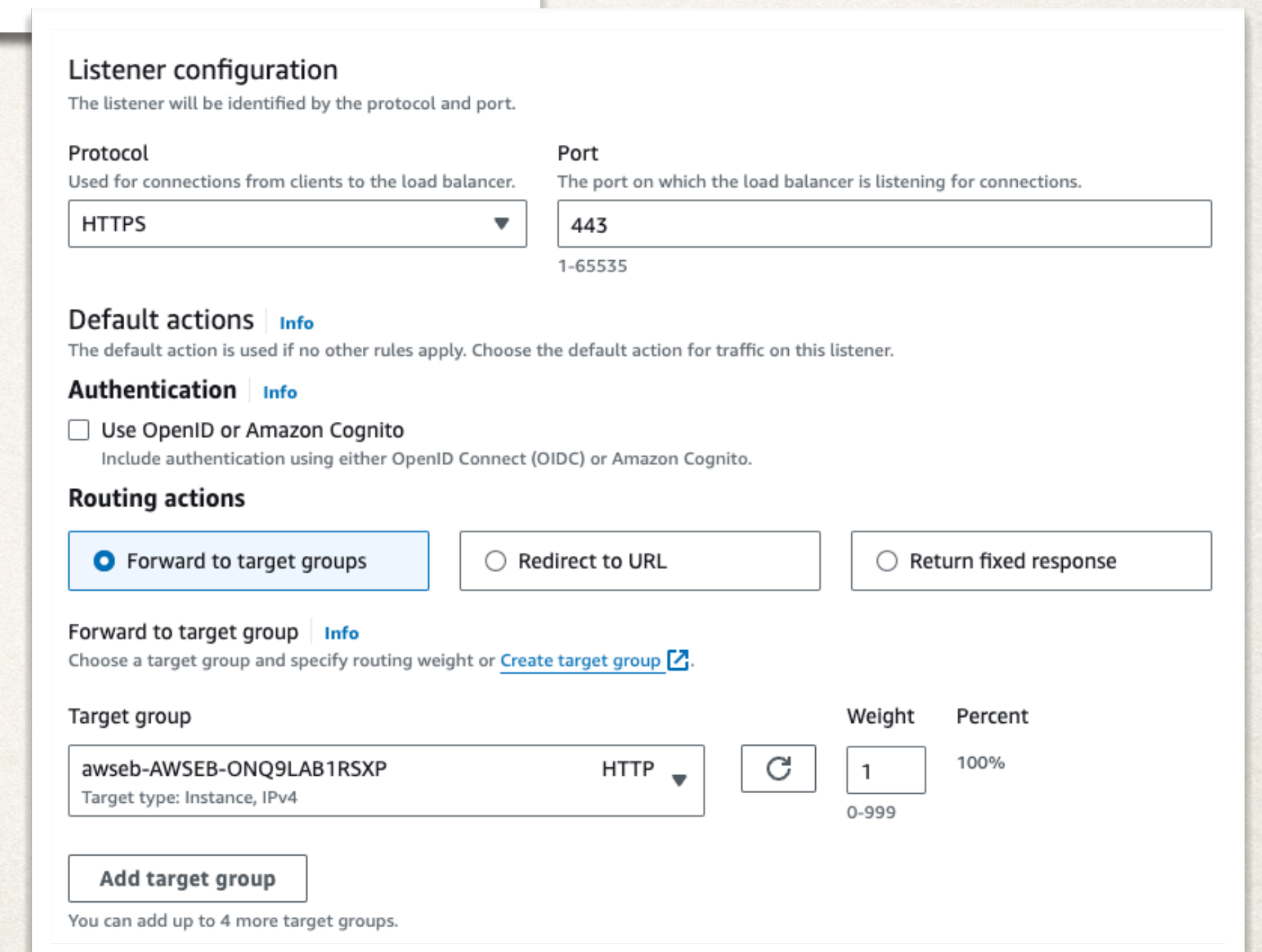
- EC2 Security Groups we need to create a new HTTPS inbound rule



The screenshot shows the 'Inbound rules' configuration for a security group. The table lists a single rule with the following details:

| Security group rule ID | Type  | Protocol | Port range | Source | Description - optional |                                        |
|------------------------|-------|----------|------------|--------|------------------------|----------------------------------------|
| sgr-0011c403ebaeb2ef8  | HTTPS | TCP      | 443        | Custom |                        | <div>0.0.0.0/0</div> <div>Delete</div> |

- Followed by a new Load Balancer listener



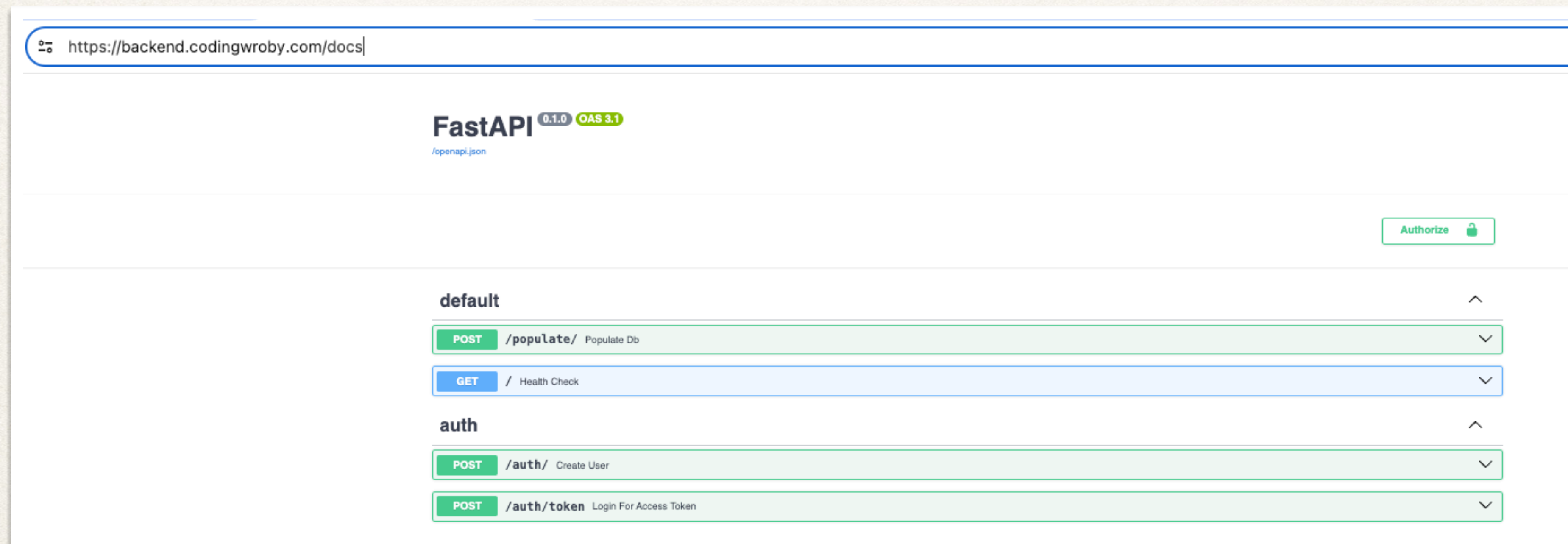
The screenshot shows the 'Listener configuration' for a new Load Balancer listener. The configuration includes:

- Protocol:** HTTPS
- Port:** 443
- Default actions:** Forward to target groups (selected)
- Authentication:** Use OpenID or Amazon Cognito (unchecked)
- Routing actions:** Forward to target group (selected)
- Target group:** awseb-AWSEB-ONQ9LAB1RSXP (Target type: Instance, IPv4)
- Weight:** 1 (Percent: 100%)



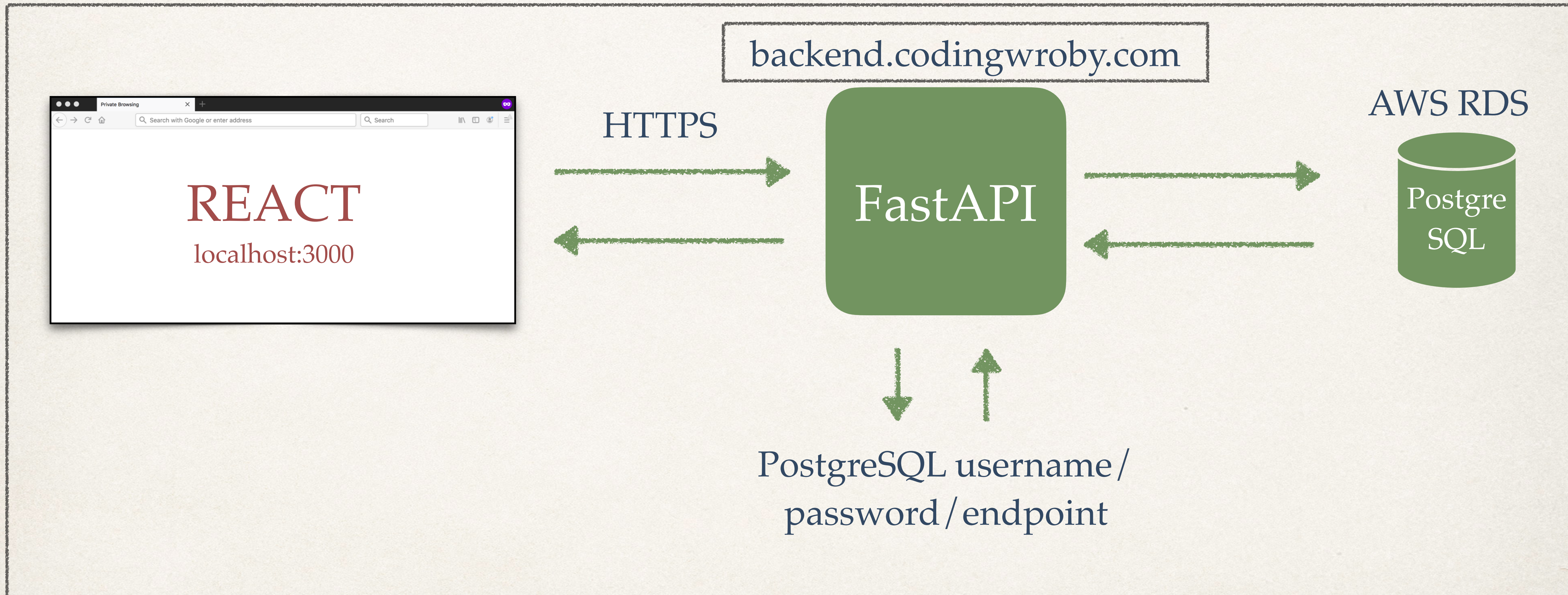
# Success!

- <https://backend.codingwroby.com> now active





# Project Overview





# Deploy React Project Overview





# AWS Core Services (Amplify)



- Deploy our SPA applications efficiently.
- Allow us to implement CI/CD for our front end application.
- Store front end environmental variables.
- Scale for users.
- Works with React.
- Include our domain



# How to implement Amplify?



- Setup our GitHub connection.

- Add env variables.

| Key               | Value                           |        |
|-------------------|---------------------------------|--------|
| REACT_APP_API_URL | https://backend.codingwroby.com | Remove |

Pointed at <https://backend.codingwroby.com>



# AWS Core Services (Amplify)



- Deploy and wait. Our app will be live.
- Example domain: `main.d2zz8pm.amplifyapp.com`
- And, we get a CORS error when running our React app on this URL.
- Hmm, how to fix?
  1. Add domain management on Amplify.
  2. Allow endpoint from `https:codingwroby.com` on FastAPI.



# AWS Core Services (Amplify)



- Add domain to domain management on Amplify.

**Domain management**

Use your own custom domain with free HTTPS to provide a secure, friendly URL for your app. Register your domain on Amazon Route53 for a one-click setup, or connect any domain registered on a 3rd party provider.

**Add domain**

Domain  
Enter the name of your root domain (eg. yourdomain.com)

Subdomains  
Configure subdomains for your app.

☒ Setup redirect from https://codingwroby.com to https://www.codingwroby.com  
You can edit these settings in the 'Rewrites and redirects' page.

Choose your certificate

☒ Amplify managed certificate

☐ Custom SSL certificate  
Manage custom SSL certificates directly on Amazon Certificates Manager. [Manage certificates](#)

- Add https://codingwroby.com as env variable on Elastic Beanstalk



# Course Summary

The logo features the text 'AWS' in white, 'FULL' in light green, and 'STACK' in light green, all in a bold, italicized sans-serif font. A white cloud icon is positioned between 'AWS' and 'FULL'. The entire logo is set against a dark green rectangular background.

***AWS***   
***FULL STACK***



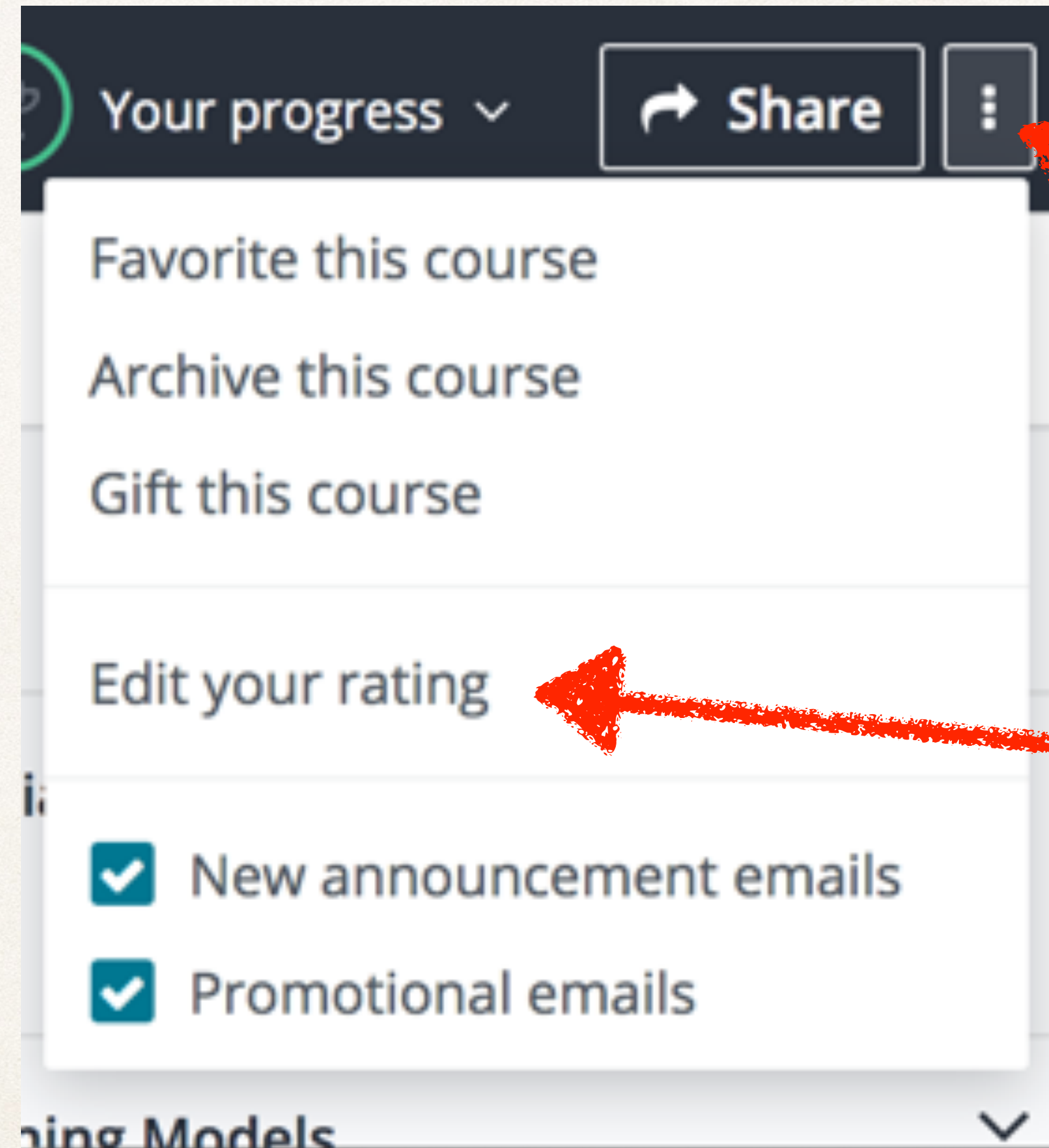
# Accomplishments

**Mission  
Accomplished**

- Deploy FastAPI + PostgreSQL application
- Setup CI/CD
- Setup Custom Domain
- Setup SSL / HTTPS
- Create Load Balancer for our FastAPI application
- Deploy React application



# Please Rate the Course





# Contact Me!

Eric Roby

[www.codingwithroby.com](http://www.codingwithroby.com)

